



3장

Storage Account 사용하기

Storage Account

Azure Storage Account란?

Azure Storage Account 사용의 장점

On-Premises Storage vs. Azure Storage

Storage Account에 제공하는 4개의 Data Storage 서비스

Containers 데이터 저장소가 제공하는 3가지 Blob 유형

Azure Storage Account 종류

Storage 복제 전략 선택하기

Storage Account

Azure Storage Account 종류

Storage 접속에 사용된 고유한 URL

Storage Explorer 도구

Storage Account Endpoint 보호하기

Storage Account의 사용자 지정 도메인 구성하기

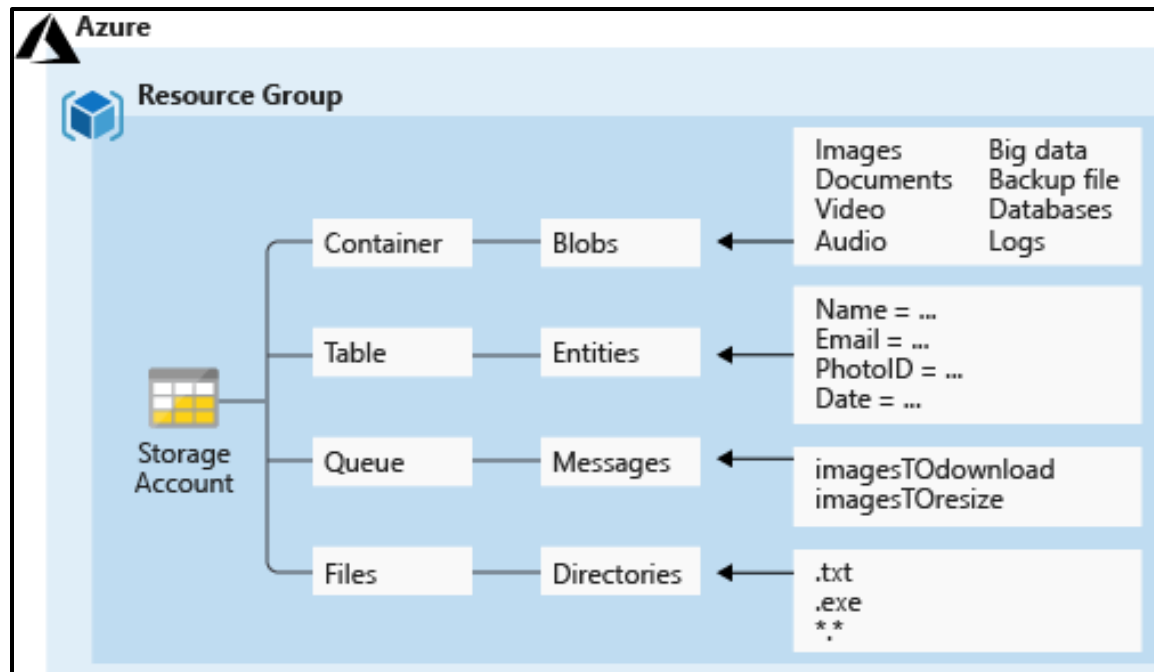
Static Website 생성하기

수명 주기 관리(Lifecycle Management) 구성하기

Storage Account

- **Azure Storage Account**란?

- On-Premises에서 운영하는 Storage를 Cloud에서도 운영하는 것
- Azure에서 데이터를 저장할 수 있는 다양한 서비스와 기능을 제공하는 기본적인 스토리지 리소스이다
- 대규모로 확장 가능하고 고가용성을 갖추고 있으며, 데이터 보안을 위한 다양한 옵션을 제공한다



Storage Account

- **Azure Storage Account** 사용의 장점

- Azure Storage Account는 On-Premises 스토리지와 비교할 때 다양한 장점과 기능을 제공하며, 클라우드 환경에서의 데이터 저장과 관리에 최적화되어 있다
- 이는 특히 확장성, 비용 효율성, 관리 용이성, 보안 측면에서 큰 이점을 제공한다
- 자동 확장
 - 사용량이 증가하면 자동으로 확장하여 추가 용량을 제공하므로, 성능 저하 없이 안정적으로 서비스를 제공할 수 있다
- 재해 복구
 - Azure는 여러 지역에 데이터를 복제하여 자연 재해나 기타 사고 발생 시에도 데이터를 안전하게 보호한다

Storage Account

- **Azure Storage Account** 사용의 장점

- 손쉬운 관리
 - **Azure Portal**을 통해 쉽게 스토리지 리소스를 관리할 수 있으며, **Azure Monitor**와 같은 도구를 통해 실시간으로 모니터링할 수 있다
- 비용 절감
 - 초기 투자 비용 없이 필요에 따라 스토리지 용량을 조정할 수 있어, 비용 효율적인 운영이 가능하다
- 보안
 - 기본 제공되는 데이터 암호화와 **Entra ID**를 통한 강력한 인증 및 권한 관리로 데이터 보안을 강화할 수 있다

Storage Account

- **On-Premises Storage vs. Azure Storage**

- Azure Storage Account는 단순히 On-Premises에 있는 스토리지를 클라우드로 옮겨 놓은 것과는 몇 가지 중요한 차이점이 있다
- Azure Storage Account는 클라우드의 특성을 활용하여 더 많은 기능과 장점을 제공하며, On-Premises 스토리지와 비교할 때 다음과 같은 차이점을 고려해야 한다
- 확장성 및 가용성
 - **Azure Storage:** 자동으로 확장 가능하며, 전 세계 데이터 센터를 통해 고가용성을 제공한다. 데이터를 지리적으로 분산시켜 재해 복구 기능을 갖추고 있다.
 - **On-Premises Storage:** 용량과 성능이 물리적 하드웨어에 의존하므로 확장이 제한적입니다. 추가 용량이 필요할 때는 새로운 하드웨어를 설치해야 합니다.
- 관리 및 유지보수
 - **Azure Storage:** 하드웨어 관리, 소프트웨어 업데이트, 백업 등의 유지보수가 필요 없다. Microsoft가 모든 관리를 대신 해준다
 - **On-Premises Storage:** 시스템 관리자나 IT 팀이 하드웨어 및 소프트웨어를 직접 관리하고 유지보수해야 한다

Storage Account

- **On-Premises Storage vs. Azure Storage**

- 비용 구조

- **Azure Storage:** 사용한 만큼 비용을 지불하는 종량제 요금제를 사용하며, 초기 투자 비용이 없다. 필요에 따라 용량을 조정할 수 있어 비용 효율적임
- **On-Premises Storage:** 초기 하드웨어 및 인프라 구축 비용이 높으며, 유지보수 비용도 발생한다. 용량 증설 시 추가 하드웨어 비용이 발생한다

- 접근성 및 통합

- **Azure Storage:** 전 세계 어디서나 인터넷을 통해 접근 가능하며, 다양한 Azure 서비스와 쉽게 통합된다. REST API를 통해 다양한 프로그래밍 언어로 접근할 수 있다.
- **On-Premises Storage:** 로컬 네트워크를 통해 접근하며, 외부에서 접근하려면 VPN이나 다른 네트워크 설정이 필요하다

- 보안 및 규정 준수

- **Azure Storage:** 기본적으로 데이터 암호화가 제공되며, 다양한 보안 규정과 표준을 준수한다. Entra ID와 같은 강력한 인증 및 권한 부여 기능을 제공한다
- **On-Premises Storage:** 보안 설정 및 규정 준수는 전적으로 조직의 책임이다. 자체적으로 보안 시스템을 구축하고 관리해야 한다

Storage Account

- 다양한 환경과 애플리케이션에서 **Azure Storage** 활용하기
 - 백업 및 복구
 - 환경: IT 인프라, 데이터베이스, 가상 머신, 파일 시스템
 - 응용: 주기적인 백업 데이터를 **Azure Blob Storage**에 저장하여 데이터 손실에 대비한 복구 시스템 구축. **Azure Backup** 서비스를 통해 자동화된 백업 및 복구 솔루션 제공
 - 데이터 아카이빙
 - 환경: 기업 데이터 센터, 연구 기관, 의료 기록 시스템
 - 응용: 오래된 데이터를 저비용으로 보관하기 위해 **Azure Cool Blob Storage**나 **Archive Blob Storage**를 사용.
 - 법적 규제 준수를 위한 장기 데이터 보관.
 - 웹 애플리케이션 호스팅
 - 환경: 웹 서버, 콘텐츠 관리 시스템(CMS)
 - 응용: 정적 웹사이트 또는 정적 파일(이미지, CSS, JavaScript)을 **Blob Storage**에 저장하여 고가용성 및 확장성을 제공
 - **Azure CDN(Content Delivery Network)**과 결합하여 전 세계적으로 빠른 콘텐츠 제공.

Storage Account

- 다양한 환경과 애플리케이션에서 **Azure Storage** 활용하기
 - 대규모 데이터 분석
 - 환경: 빅 데이터 분석 플랫폼, 데이터 웨어하우스
 - 응용: **Azure Data Lake Storage**를 사용하여 대규모 데이터를 저장하고 분석. **Azure Databricks**, **HDInsight**, **Azure Synapse Analytics**와 통합하여 데이터 처리 및 분석.
 - **IoT** 애플리케이션
 - 환경: **IoT** 장치, 센서 네트워크
 - 응용: **IoT** 디바이스에서 생성된 데이터를 **Queue Storage**나 **Blob Storage**에 저장하여 실시간 또는 배치 처리. **Azure IoT Hub**와 통합하여 데이터 수집 및 분석.
 - 미디어 및 엔터테인먼트
 - 환경: 비디오 스트리밍 서비스, 미디어 아카이브
 - 응용: 대용량 비디오 파일, 이미지 파일 등을 **Blob Storage**에 저장하고, **Azure Media Services**와 통합하여 비디오 인코딩 및 스트리밍.
 - 애플리케이션 데이터 저장소
 - 환경: 모바일 애플리케이션, 웹 애플리케이션
 - 응용: 애플리케이션 데이터(예: 사용자 파일, 로그 데이터)를 **Blob Storage**에 저장. **Azure Files**를 사용하여 파일 시스템처럼 접근 가능.

Storage Account

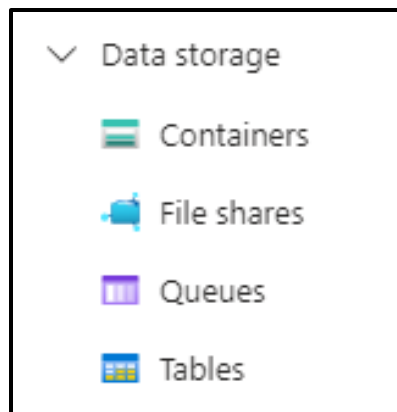
- 다양한 환경과 애플리케이션에서 **Azure Storage** 활용하기
 - 분산 애플리케이션 메시지 큐
 - 환경: 분산 시스템, 마이크로서비스 아키텍처
 - 응용: **Queue Storage**를 사용하여 애플리케이션 간 비동기 메시지 전달 및 작업 큐 관리. **Azure Service Bus**와 통합하여 복잡한 메시징 요구 사항 처리
 - **CI/CD** 파이프라인 **Artifact** 저장소
 - 환경: **DevOps** 환경, 빌드 및 배포 시스템
 - 응용: 빌드 아티팩트, **Docker** 이미지 등을 **Blob Storage**에 저장하여 **CI/CD** 파이프라인에서 활용. **Azure DevOps**와 통합하여 자동화된 배포 프로세스 구현.
 - 기계 학습 및 **AI**
 - 환경: 머신러닝 워크플로우, 데이터 사이언스 프로젝트
 - 응용: 대규모 학습 데이터셋을 **Blob Storage**에 저장하여 **Azure Machine Learning** 서비스와 통합. 모델 학습 및 평가 데이터 저장.

Storage Account

- **Storage Account**에 제공하는 **4개의 Data Storage** 서비스

- **Containers**

- 비정형 데이터를 저장하는 데 사용된다. 대용량 파일, 이미지, 비디오 등을 저장할 수 있다
- **Blob**은 크게 **Block Blob, Append Blob, Page Blob**으로 나뉜다
- 예시: 대규모 로그 파일 저장, 백업 데이터 저장



- **File Shares**

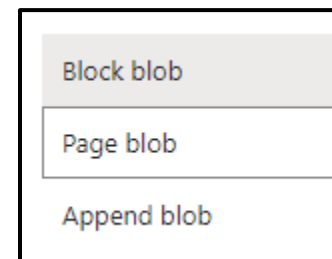
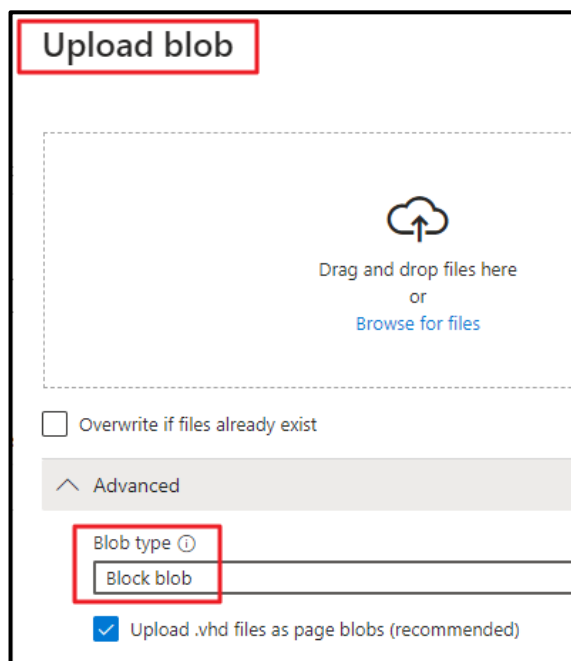
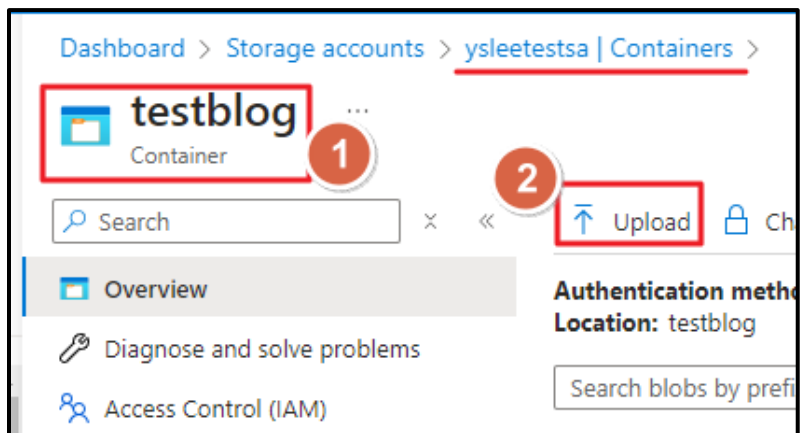
- 완전 관리형 파일 공유를 제공하며, SMB (Server Message Block) 프로토콜을 통해 접근 가능하다
- 예시: 기업 내부 파일 공유, 애플리케이션에서 파일 시스템으로 사용.

Storage Account

- **Storage Account**에 제공하는 **4개의 Data Storage** 서비스
 - **Queues**
 - 대규모 메시지 큐를 제공한다. 애플리케이션 구성 요소 간에 메시지를 비동기적으로 전달하는 데 사용된다
 - 예시: 주문 처리 시스템에서 작업을 비동기적으로 처리하기 위해 큐를 사용.
 - **Tables**
 - 비관계형(NoSQL) 데이터 저장소로, 대규모 반구조화된 데이터를 저장할 수 있다
 - 예시: 사용자 프로필 정보, 애플리케이션 이벤트 로그 저장.

Storage Account

- **Containers** 데이터 저장소가 제공하는 **3가지 Blob** 유형
 - Container에 파일을 업로드할 때 세 가지 유형의 **Blob**, 즉 **Block Blob**, **Append Blob**, **Page Blob**을 선택하여 업로드해야 한다
 - 각 **Blob** 유형은 특정한 사용 사례와 요구 사항에 맞춰 설계되어 있기 때문에 애플리케이션의 특성과 저장해야 할 데이터의 유형에 따라 적절한 **Blob** 유형을 선택하는 것이 중요하다



Storage Account

- **Containers** 데이터 저장소가 제공하는 **3가지 Blob** 유형
 - **Block Blob**
 - 웹 애플리케이션에서 사용자 프로필 사진을 업로드하고 저장
 - 비디오 스트리밍 서비스에서 비디오 파일을 저장하여 사용자에게 제공
 - **Append Blob**
 - 웹 서버가 애플리케이션 로그를 지속적으로 기록하고 저장.
 - IoT 디바이스에서 실시간 센서 데이터를 수집하여 저장
 - **Page Blob**
 - **Virtual Machine**의 운영 체제 디스크를 저장하여 가상 머신을 부팅 및 실행
 - 데이터베이스 서버에서 데이터 파일을 저장하여 빠른 읽기/쓰기 성능을 제공.

Storage Account

- **Azure Storage Account** 종류

- **Storage Account**를 생성할 때 **Storage의 Performance에 따라** Storage Account 종류를 선택한다
 - **Standard** (##일반적으로 많이 사용)
 - **Premium**

The screenshot shows the 'Instance details' section of the Azure Storage Account creation wizard. The 'Storage account name' is 'ktds2024', the 'Region' is '(US) East US', and the 'Performance' is set to 'Standard'. The 'Premium account type' dropdown is open, showing options for 'Page blobs', 'Block blobs', and 'File shares'. The 'Redundancy' is set to 'LRS'.

Subscription * MSDN Platforms

Resource group * testRG
[Create new](#)

Instance details

Storage account name * ⓘ ktds2024

Region * ⓘ (US) East US
[Deploy to an Azure Extended Zone](#)

Performance * ⓘ

1 ☐ Standard: Recommended for most scenarios (general-purpose v2 account)

☒ Premium: Recommended for scenarios that require low latency.

Premium account type * ⓘ

2

Page blobs

Block blobs:
Best for high transaction rates or low storage latency

File shares:
Best for enterprise or high-performance applications that need to scale

Page blobs:
Best for random read and write operations

Redundancy * ⓘ LRS

Previous Next Review & Create

Storage Account

- **Azure Storage Account** 종류
 - **Standard general-purpose v2**
 - 가장 최신의 일반 목적 스토리지 계정으로, 모든 **Azure Storage** 기능을 지원
 - Blob Storage, File Storage, Queue Storage, Table Storage, Disk Storage 모두 지원
 - 비용 효율적이며, 다양한 액세스 계층(Hot, Cool, Archive)을 지원하여 데이터 사용 패턴에 따라 비용을 최적화할 수 있다
 - 자동으로 확장되며, 높은 가용성과 내구성을 제공
 - 사용 사례:
 - 다양한 데이터 저장 요구가 있는 애플리케이션
 - 비정형 데이터 저장(이미지, 동영상, 문서)
 - 주기적으로 접근하지 않는 데이터의 저비용 저장.

Storage Account

- **Azure Storage Account** 종류

- **Premium block blobs**

- 높은 성능이 요구되는 워크로드를 위해 설계된 프리미엄 블록 **Blob** 스토리지 계정
- 낮은 지연 시간과 높은 트랜잭션 속도를 제공
- 일반적으로 높은 **I/O** 성능이 요구되는 데이터 저장에 사용
- 비용이 상대적으로 높지만, 성능이 중요할 때 유리
- 사용 사례:
 - 미디어 처리 애플리케이션(예: 동영상 인코딩)
 - 고성능 분석 애플리케이션
 - 실시간 데이터 처리 요구가 있는 시나리오.

Storage Account

- **Azure Storage Account** 종류

- **Premium file shares**

- 고성능이 요구되는 파일 공유 서비스를 제공하는 프리미엄 파일 스토리지 계정
- 낮은 지연 시간과 높은 처리량을 제공
- **SMB** 프로토콜을 통해 접근 가능하며, 대규모 파일 공유가 필요한 애플리케이션에 적합
- 일반적인 파일 공유보다 높은 성능을 제공
- 사용 사례:
 - 엔터프라이즈 파일 공유 및 협업 도구
 - 데이터베이스 백업 및 복구
 - 애플리케이션 개발 환경에서의 고성능 파일 액세스.

Storage Account

- **Azure Storage Account** 종류
 - **Premium page blobs**
 - **Azure Virtual Machine**의 디스크 저장소로 사용되는 프리미엄 페이지 Blob 스토리지 계정
 - 낮은 지연 시간과 높은 처리량을 제공하여 **VM** 디스크 스토리지에 최적화
 - 주로 **OS** 디스크와 데이터 디스크로 사용
 - 안정적인 성능과 높은 IOPS(Input/Output Operations Per Second)를 제공
 - 사용 사례:
 - **Azure Virtual Machine**의 운영 체제 디스크 및 데이터 디스크
 - 고성능 데이터베이스 파일 저장
 - 자주 업데이트되는 임의 접근 파일(예: 캐시 파일).

Storage Account

- **Storage** 복제 전략 선택하기
 - Storage Account를 생성할 때 Data 복제를 어떻게 할지 결정할 수 있다

Performance * ⓘ

☒ Standard: Recommended for most scenarios (general-purpose v2 account)

☐ Premium: Recommended for scenarios that require low latency.

Redundancy * ⓘ

Geo-redundant storage (GRS) ▼

☒ Make read access to data available in the event of regional unavailability.

Locally-redundant storage (LRS):
Lowest-cost option with basic protection against server rack and drive failures.
Recommended for non-critical scenarios.

Geo-redundant storage (GRS):
Intermediate option with failover capabilities in a secondary region.
Recommended for backup scenarios.

Zone-redundant storage (ZRS):
Intermediate option with protection against datacenter-level failures.
Recommended for high availability scenarios.

Geo-zone-redundant storage (GZRS):
Optimal data protection solution that includes the offerings of both GRS and ZRS. Recommended for critical data scenarios.

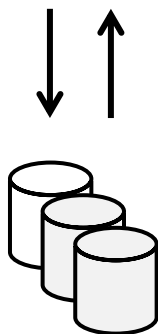
Geo-redundant storage (GRS) ▼

☒ Make read access to data available in the event of regional unavailability.

Storage Account

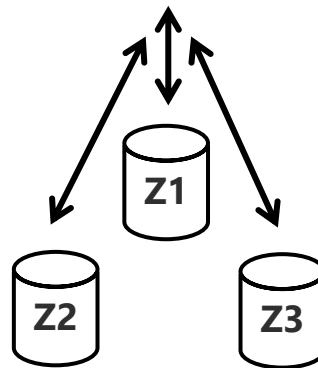
- **Storage** 복제 전략 선택하기

단일 지역



LRS

- 복제본 3개, 지역 1개
- 디스크, 노드, 랙 오류로부터 보호
- 모든 복제본이 커밋될 때 쓰기가 승인됩니다.
- 이중 패리티 RAID보다 우수



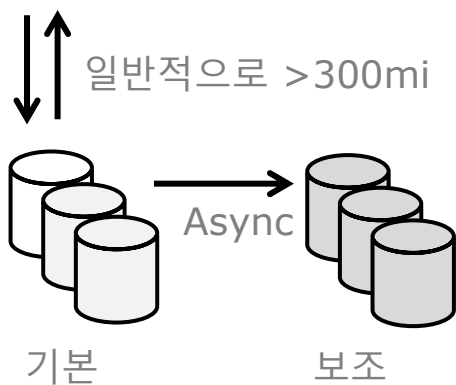
ZRS

- 복제본 3개, 영역 3개, 지역 1개
- 디스크, 노드, 랙 및 영역 오류로부터 보호
- 세 영역 모두에 동기 쓰기

Storage Account

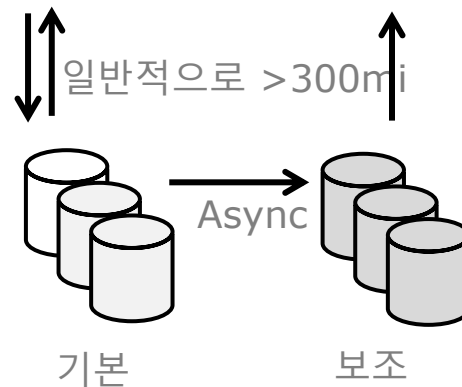
- **Storage** 복제 전략 선택하기

여러 지역



GRS

- 복제본 6개, 지역 2개(지역당 3개)
- 주요 지역 재해로부터 보호
- 보조에 대한 비동기 복사본



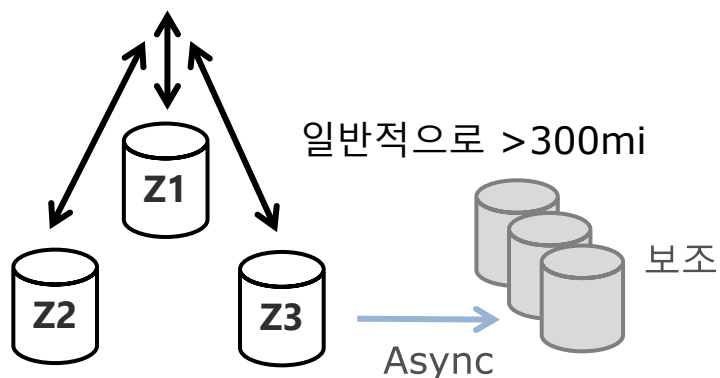
RA-GRS

- GRS + 보조에 대한 읽기 액세스
- 별도의 보조 엔드포인트
- RPO(복구 지점 목표) 지연을 보조로 쿼리할 수 있음

Storage Account

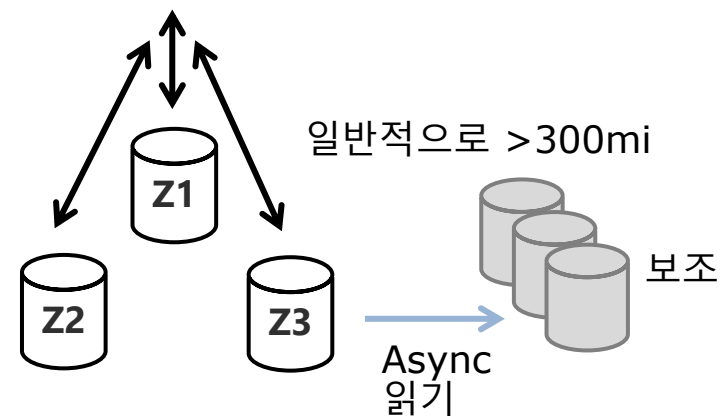
- **Storage** 복제 전략 선택하기

여러 지역



GZRS

- 복제본 6개, 영역 3+1개, 지역 2개
- 디스크, 노드, 랙, 영역 및 지역 오류로부터 보호
- 세 개 영역 모두에 동기식 쓰기 그리고 보조에 비동기식 쓰기



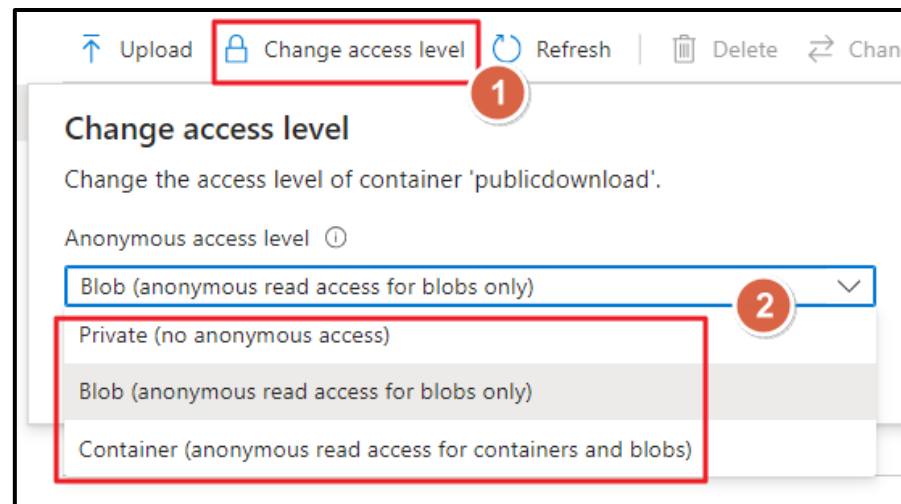
RA-GZRS

- GZRS + 보조에 대한 읽기 액세스
- 별도의 보조 엔드포인트
- 보조에 대한 RPO 지연을 쿼리할 수 있습니다.

Storage Account

- **Container의 Access Level** 변경하기

- Storage Account를 생성한 후 Blob에 Container를 생성한 후 Access Level을 필요에 따라 변경할 수 있다
- **Private**
 - Azure에 로그인한 권한을 가진 사람만 파일을 업로드/다운로드할 수 있다
 - 만약에 익명 사용자에게 링크를 공유하여 다운로드하게 하고자 하면 해당 파일에 SAS를 생성하여 해당 URL을 공유하면 가능하다
- **Blob**
 - Private과 동일하며 익명 사용자도 URL 링크만 있으면 다운로드 가능하다(##강추)
- **Container**
 - Blob과 동일하며 REST API로 다양한 작업을 할 수 있다



Storage Account

- **Container의 Access Level 변경하기**

- **Access Level을 Container를 사용하는 경우**

- 애플리케이션에서 해당 컨테이너에 저장된 **Blob** 파일에 접근하여 사용하도록 설정하면 애플리케이션이 별도의 인증 절차 없이도 **URL**을 통해 컨테이너의 모든 파일에 접근할 수 있게 된다

1. 정적 파일 접근

- 웹 애플리케이션에서 정적 자산(이미지, CSS, JavaScript 파일 등)을 **Azure Blob Storage**에 저장하고, 이 파일들을 **URL**로 직접 호출하여 웹 페이지에 로드한다. 이 경우, **Blob** 파일의 **URL**을 **HTML** 코드나 **CSS**, **JS** 파일에 직접 포함할 수 있다

```
  
<link rel="stylesheet" href="https://<storage_account>.blob.core.windows.net/<container_name>/style.css">
```

2. 파일 다운로드

- 애플리케이션에서 사용자에게 파일을 다운로드할 수 있는 링크를 제공한다. 이 경우 사용자는 클릭만으로 **Blob**에 저장된 파일을 다운로드할 수 있다

```
<a href="https://<storage_account>.blob.core.windows.net/<container_name>/document.pdf"  
download>Download Document</a>
```

3. **API**와 연동

- 서버 측 애플리케이션에서 **Azure Blob Storage**에 저장된 데이터를 **API**의 응답으로 제공할 수 있다. 이를 통해 클라이언트 애플리케이션이 **Blob** 스토리지에서 직접 데이터를 가져가도록 할 수 있다.

```
{
```

Storage Account

- **Container의 Access Level** 변경하기
 - **Access Level**을 **Container**를 사용하는 경우
 - 특정한 **Container**에 있는 **Blob** 목록 조회하기
 - https://<storage_account>.blob.core.windows.net/<container_name>?restype=container&comp=list

This XML file does not appear to have any style information associated with it. The document tree is shown below.

```
<?xml version="1.0" encoding="utf-8"?>
<EnumerationResults ContainerName="https://<storage_account>.blob.core.windows.net/download">
  <Blobs>
    <Blob>
      <Name>1723640913360-windows_server_2022_logo-removebg-preview.png</Name>
      <Url>https://<storage_account>.blob.core.windows.net/download/1723640913360-windows_server_2022_logo-removebg-preview.png</Url>
      <Properties>
        <Last-Modified>Wed, 14 Aug 2024 13:08:47 GMT</Last-Modified>
        <Etag>0x8DCBC623BA17BA8</Etag>
        <Content-Length>3678</Content-Length>
        <Content-Type>image/png</Content-Type>
        <Content-Encoding/>
        <Content-Language/>
        <Content-MD5>RAkwSbB/ijZyc150ieHqCw==</Content-MD5>
        <Cache-Control/>
        <BlobType>BlockBlob</BlobType>
        <LeaseStatus>unlocked</LeaseStatus>
      </Properties>
    </Blob>
    <Blob>
      <Name>1723642811674-2019년가정사역팀-평화로운엄마아빠.jpg</Name>
      <Url>https://<storage_account>.blob.core.windows.net/download/1723642811674-2019년가정사역팀-평화로운엄마아빠.jpg</Url>
      <Properties>
        <Last-Modified>Wed, 14 Aug 2024 13:40:25 GMT</Last-Modified>
        <Etag>0x8DCBC66A73108B4</Etag>
        <Content-Length>977160</Content-Length>
```

Demo-Storage Account

• Demo-Container Data Storage 사용

- Storage Account 생성하기
 - 이름: 중복되지 않은 고유한 소문자와 영문자만 사용가능
- Container 생성하기
 - 이름: down
- 파일 업로드 및 다운로드하기
- 사진, 음악, 영상 파일 읽기
- 주의 사항
 - 기본적으로 익명 접속이 제한되어 있음
 - 이것을 **Enabled**로 한 후 진행할 것



Demo-Storage Account

- **Demo-Container Data Storage** 사용

- 긴 Storage Account 이름을 Custom Domain으로 줄여서 사용하기
 - Storage Account도 하나의 컴퓨터이기 때문에 DNS에서 별칭을 사용할 수 있다
 - **nslookup** ysleepublicshare.blob.core.windows.net (##IP 주소가 있다)
 - ysleepublicshare.blob.core.windows.net을 dl.cloudshell.kr로 줄일 수 있다

The screenshot shows the 'Add record set' dialog box with the following fields:

- Name:** dl
- Type:** CNAME - Link your subdomain to another record
- Alias record set:** No
- TTL:** 1
- TTL unit:** Hours
- Alias:** ysleepublicshare.blob.core.windows.net

- 파일을 다운로드하기 위해
<https://ysleepublicshare.blob.core.windows.net/down/image.png> 대신
<http://dl.cloudshell.kr/down/image.png> 로 줄여서 사용할 수 있다

Demo-Storage Account

- **Demo-File Shares** 사용

- **File Share**를 생성할 때 마지막 단계에서 자동으로 **Recovery Service Vault** 서비스에 **Backup**되도록 되어 있다
 - 필요 없으면 이 기능을 **Uncheck**한다
- 반드시 **VM**를 만들어서 접근해야 한다
 - 물리적인 한국에서는 접근 불가하지만(**SMB 445**번 차단) **VM**을 **Korea Central, Korea South**에 있으면 접근할 수 있다
 - **Windows VM** 2개 생성하여 데모
- **Connect**를 클릭하여 **Windows** 용 접속 스크립트를 복사한다
- **VM**에서 복사한 것을 **PowerShell**에서 붙여 넣어 실행하면 **Z:** 드라이브가 생긴다
- **Network** 드라이브 연결하여 파일 공유하기

Storage Account

- **Storage** 접속에 사용된 고유한 **URL**

- 모든 개체에는 계정 이름과 스토리지 유형에 따른 고유한 **URL** 주소가 있다

Container service: <http://mystorageaccount.blob.core.windows.net>

Table service: <http://mystorageaccount.table.core.windows.net>

Queue service: <http://mystorageaccount.queue.core.windows.net>

File service: <http://mystorageaccount.file.core.windows.net>

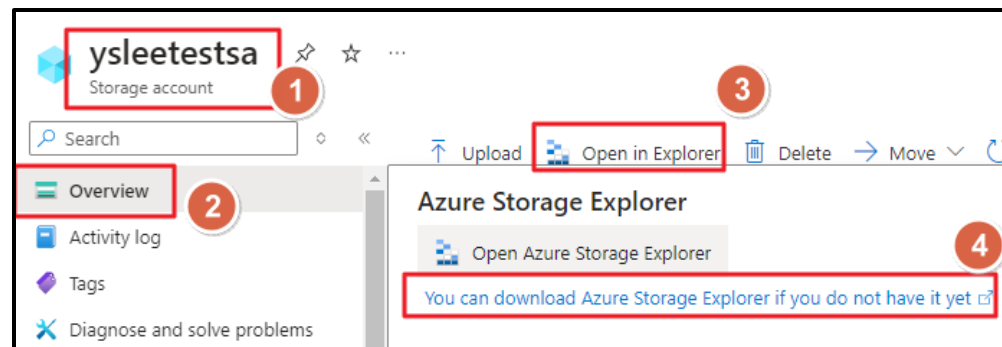
- 원할 경우, 사용자 지정 도메인 이름을 구성할 수 있다

CNAME 레코드	대상
blobs.contoso.com	contosoblobs.blob.core.windows.net

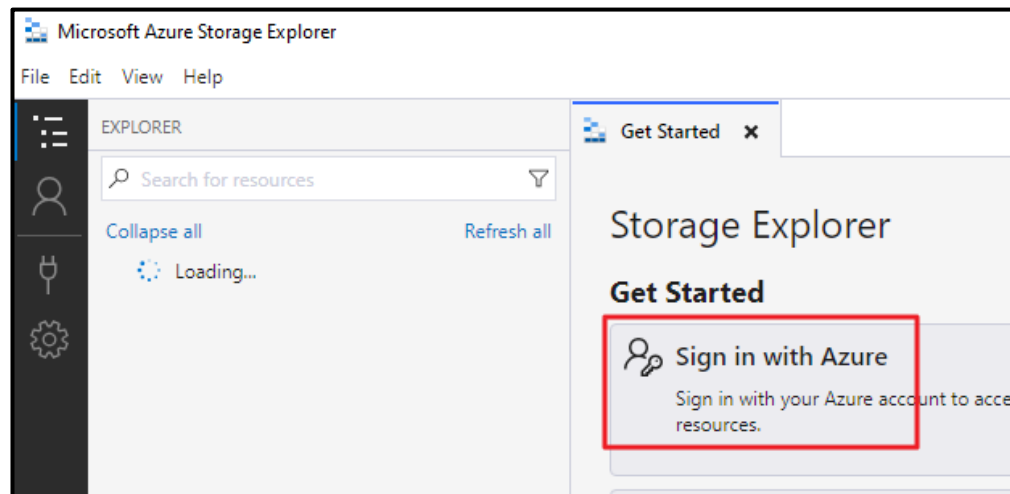
Storage Account

- **Storage Explorer** 도구

- 로컬 컴퓨터에 설치하여 **Azure Storage Account**에 파일을 업로드/다운로드하는 도구
- 프로그램 설치



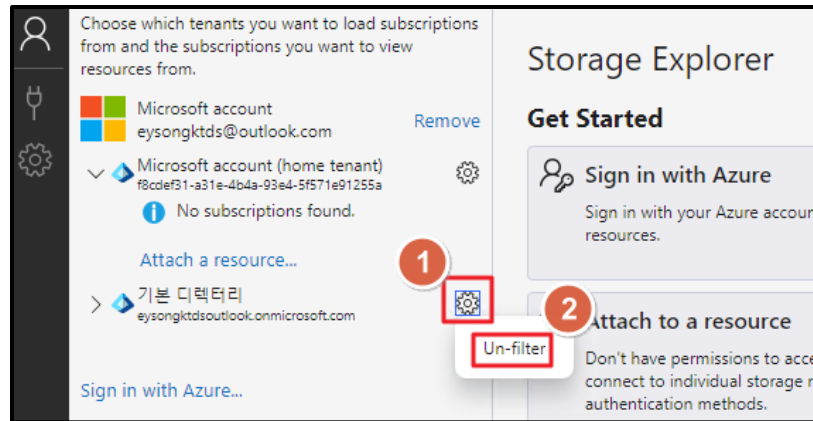
- Azure에 로그인



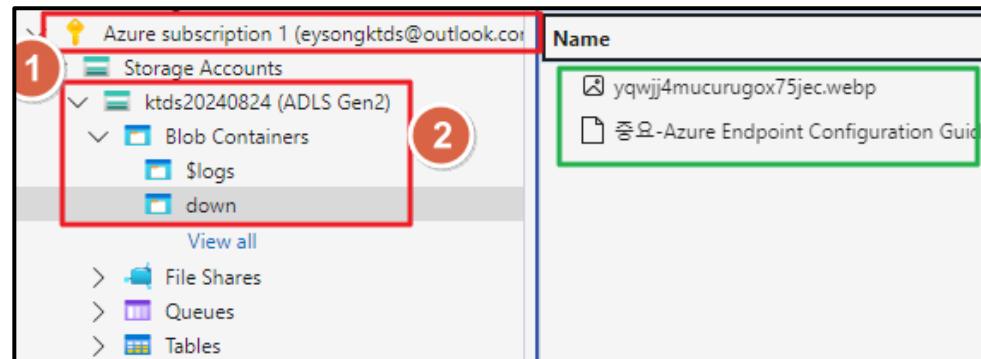
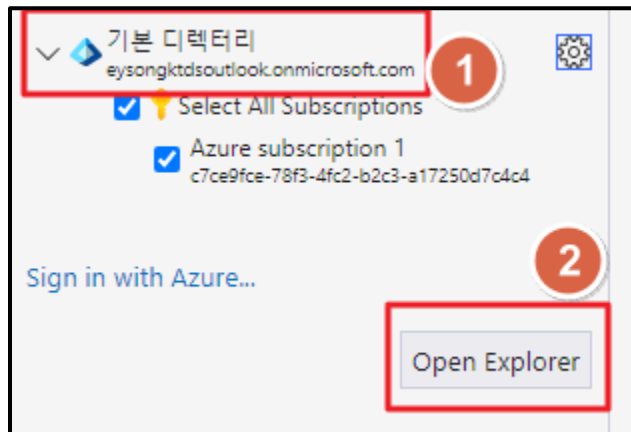
Storage Account

- **Storage Explorer** 도구

- Filter 해제

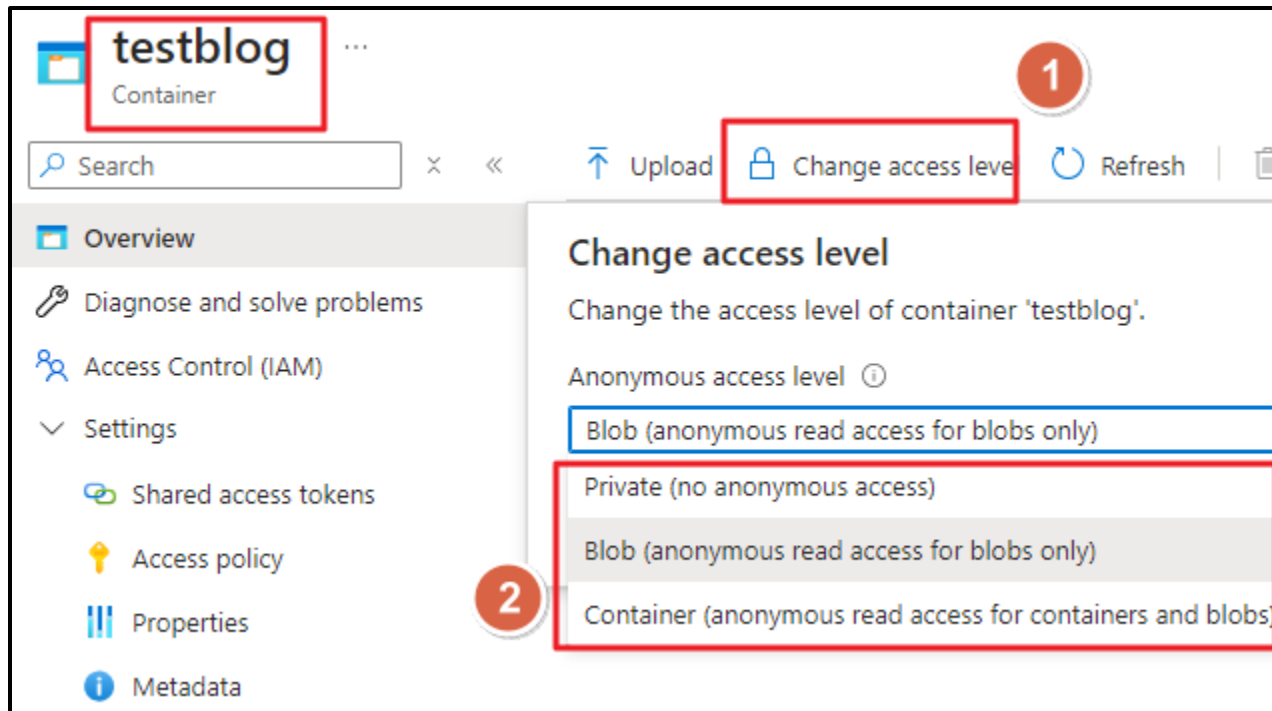


- Explorer 열기



Storage Account

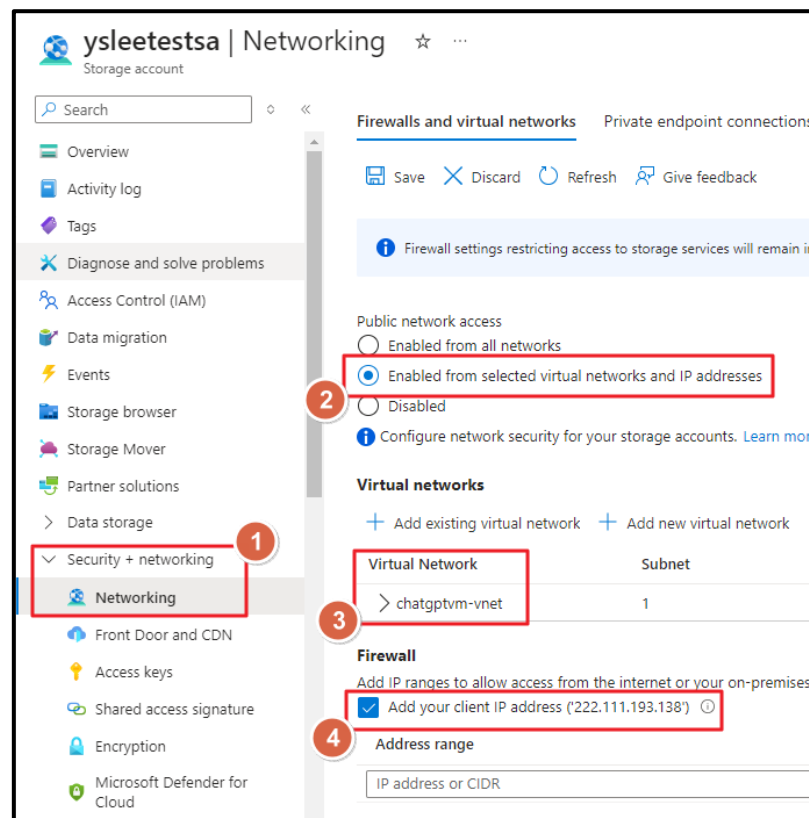
- **Container** 접속 제한하기
 - 모든 **Blob**은 컨테이너에 있어야 합니다.
 - **Private** (익명 액세스 불가능)
 - **Blob**(익명 액세스 가능)
 - **Container**
 - Restful API로 접속하여만 **Container** 목록을 확인할 수 있다



Storage Account

- **Storage Account Endpoint** 보호하기

- Firewall 및 Virtual Network를 통해 **Virtual Network**의 특정 **Subnet**이나 **Public IP**에서 **Storage** 계정에 액세스하는 것을 제한할 수 있다
- Subnet 및 Virtual Network는 Storage 계정과 동일한 Azure 지역 또는 지역 쌍에 존재해야 한다

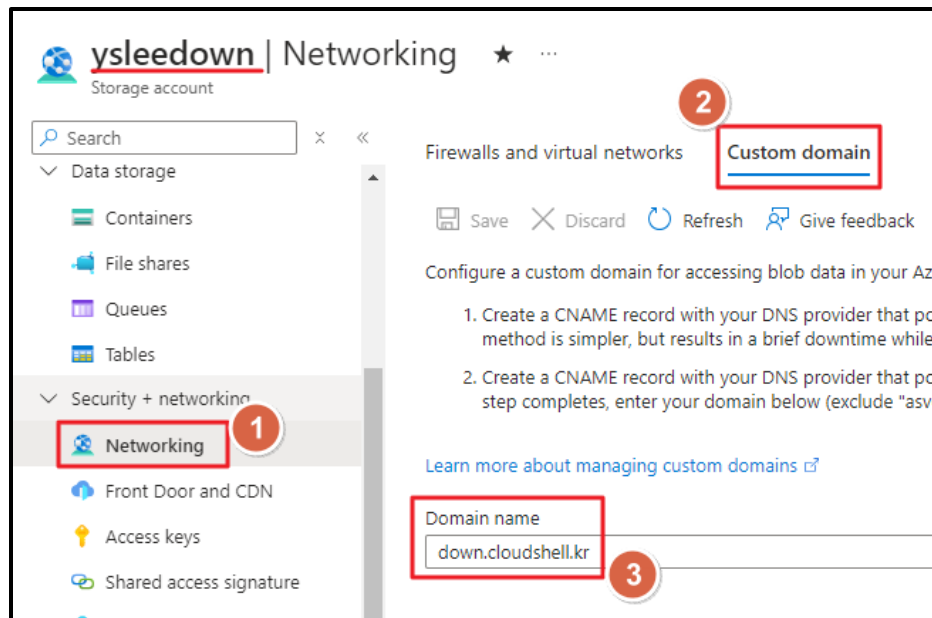


Storage Account

• Storage Account의 Custom Domain 구성하기

- 다음과 같은 순서로 진행한다

1. **Custom Domain** 준비: 도메인 등록 대행 기관에서 Domain 구입 및 DNS 관리
2. **Azure Storage** 계정 생성: Storage 계정을 생성하거나 기존 계정 사용
3. **Custom Domain** 설정: Azure Storage 계정에서 Custom domain 추가(down.cloudshell.kr)
4. **DNS** 설정 구성: 도메인 등록 대행 기관에서 CNAME 레코드 추가
5. **HTTPS** 설정 (선택 사항): Azure CDN을 통해 사용자 지정 도메인에 HTTPS 설정



Storage Account

- **Storage Account의 Custom Domain** 구성하기
 - **DNS** 관리에서 작업: 도메인 등록 대행 기관에서 **CNAME** 레코드 추가

Add record set
cloudshell.kr

1 Name: down

2 Type: CNAME - Link your subdomain to another record

Alias record set ⓘ: No

TTL *: 1

TTL unit: Hours

3 Alias: ysleedown.blob.core.windows.net

cloudshell.kr | Recordsets
DNS zone

Search

+ Add Refresh Delete

Overview
Activity log
Access control (IAM)
Tags
Diagnose and solve problems
Settings
Properties
Locks
DNS Management
Recordsets

Name	Type
down	CNAME

Showing 1 record set(s) out of total of 70 record set(s)

- 제대로 추가되었는지 확인하기
 - **nslookup** down.cloudshell.kr

```
관리자: Windows PowerShell
jesuswithme cd\
C:\> nslookup down.cloudshell.kr
서버: kns.kornet.net
Address: 168.126.63.1

권한 없는 응답:
이름: blob.se1prdstr06a.store.core.windows.net
Address: 20.60.200.161
Aliases: down.cloudshell.kr
ysleedown.blob.core.windows.net
```

Storage Account

- **Storage Account**의 **Custom Domain** 구성하기
 - Custom Domain 이름의 URL로 접속하기
 - Blob Container 생성하기
 - **media** (Access level: **Blob**)
 - 로컬 파일을 Storage Account로 업로드하기
 - 이평화.**mp4**
 - Custom Domain 이름의 URL로 업로드한 파일에 접속하기
 - <http://down.cloudshell.kr/media/이평화.mp4>



Storage Account

- 웹 서비스를 하기 위한 준비 단계
 - 개발자가 자신이 개발한 **HTML, CSS, JavaScript** 코드를 사용하여 물리적 컴퓨터를 통해 웹 서비스를 제공하려면 준비할 것이 있다
 1. 서버 컴퓨터 준비
 - 인터넷에 연결된 물리적 서버
 - 서버 운영을 위한 운영 체제(OS)가 필요(라이선스까지)
 2. 웹 서버 소프트웨어 설치 및 구성하기
 - **Apache, Nginx**, 또는 **Microsoft IIS**와 같은 웹 서버 소프트웨어를 설치
 - 웹 서버 소프트웨어의 설정 파일을 수정하여 웹 사이트가 올바르게 동작하도록 구성
 3. 도메인 이름 설정
 - 인터넷 상에서 쉽게 접근할 수 있도록 도메인 이름을 등록
 - 해당 도메인을 관리하기 위한 **DNS Server** 운영
 4. 방화벽 및 네트워크 설정
 - 방화벽에서 **80/443** 포트를 허용
 - 고정 **Public IP**가 필요

Storage Account

- 웹 서비스를 하기 위한 준비 단계

- 5. 웹 사이트 파일 배포

- 개발한 **HTML, CSS, JavaScript** 파일을 웹 서버의 루트 디렉토리에 저장
 - **Linux** 서버의 경우, 웹 서버가 파일을 읽고 제공할 수 있도록 적절한 파일 권한을 설정

- 6. **SSL** 인증서 설정 (**HTTPS** 활성화)

- 웹 사이트의 보안을 강화하기 위해 **SSL** 인증서를 구매하거나 **Let's Encrypt**와 같은 무료 인증서를 사용할 수 있다
 - 이 인증서를 웹 서버에 설치하여 **HTTPS**를 활성화한다

- 7. 테스트 및 유지 관리

- 외부에서 웹 사이트에 접근하여 모든 기능이 제대로 동작하는지 테스트
 - 서버의 상태를 모니터링하고, 운영 체제 및 웹 서버 소프트웨어의 보안 업데이트를 주기적으로 적용

- 8. 백업 및 복구 계획

- 웹 서버의 데이터를 정기적으로 백업하여 데이터 손실에 대비
 - 문제가 발생했을 때 빠르게 복구할 수 있도록 계획을 세운다

Storage Account

- **Static Website** 생성하기

- 주로 정적 콘텐츠(HTML, CSS, JavaScript, 이미지 등)를 호스팅하는 데 사용된다. 이 방법은 매우 저렴하고 간단하게 웹 사이트를 호스팅할 수 있는 옵션이다
- 주요 특징
 - 저렴한 비용: **Storage Account**의 정적 웹 사이트 기능은 매우 저렴하며, 트래픽과 스토리지 사용량에 따라 비용이 결정된다
 - 간단한 설정: 몇 가지 설정만으로 간단하게 정적 웹 사이트를 배포할 수 있다.
 - 글로벌 분산 및고가용성: **Azure CDN**과 연동하면 전 세계적으로 콘텐츠를 캐시하여 빠르게 제공할 수 있다
 - 기본적인 기능 제공: **URL** 리디렉션, 기본적인 **HTTP** 헤더 설정, **404** 에러 페이지 등을 설정할 수 있다
 - **Serverless** 환경: 웹 서버가 필요 없으며, 순수한 정적 파일만 제공하는 서버리스 방식
- 사용 사례
 - 간단한 포트폴리오 웹 사이트
 - 마케팅 랜딩 페이지
 - 정적 콘텐츠 제공 (예: 문서, 이미지)

Storage Account

- **Demo-Static Website** 생성하기

- **Primary Endpoint** 생성하기

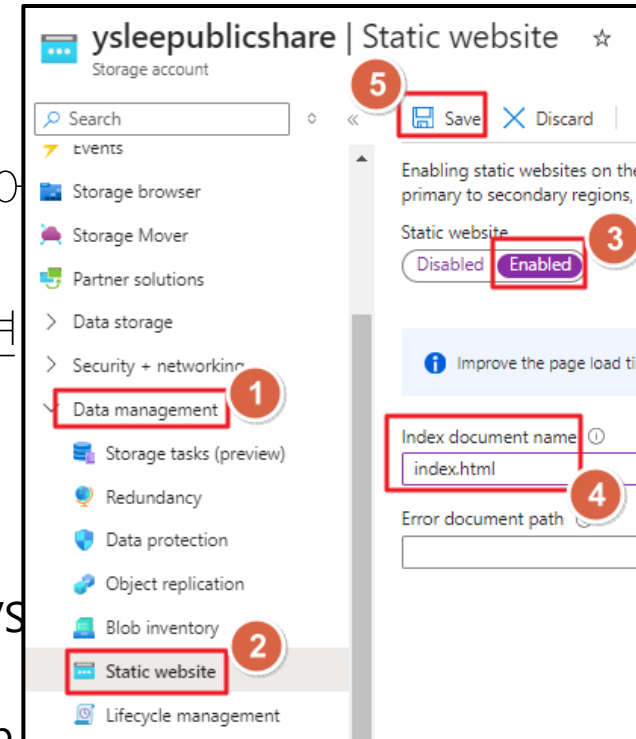
- <https://ysleepublicshare.z12.web.core.windows.net/>

- 생성된 URL로 접속하거나 또는 QR Code를 만들어 생성할 수도 있다

- 이런 후에 Data Storage - Containers로 이동하면 **\$web**이라는 Container가 생성되어 있는데, 이곳에 **index.html** 파일을 업로드한다

- 그런 다음에 비로소 접속할 수 있다

- index.html 파일은 notepad.exe나 다른 편집 프로그램(VS Code)으로 만들 수 있다
- chatgpt.com이나 claude.ai에서도 자연어 질문으로 Web App을 생성할 수 있다



Storage Account

- **Demo-Static Website** 생성하기

- **HTML** 메타 태그나 **JavaScript**를 사용하여 **URL Redirection**하기

- index.html 파일 내용을 다음과 같이 작업하면 <http://powershell.kr>로 자동으로 이동할 수 있다
- <https://ysleepublicshare.z12.web.core.windows.net/> 로 접속하면 다른 Web Site로 보낼 수 있다는 뜻

```
<!DOCTYPE html>
<html>
<head>
  <meta http-equiv="refresh" content="0; url=http://powershell.kr/" />
  <title>Redirecting...</title>
</head>
<body>
  <p>If you are not redirected automatically, follow this <a href="https://powershell.kr/">link</a>.</p>
</body>
</html>
```

- 가능하면 QR Code로 접속하는 것이 좋을 것 같다
- Static Website는 DNS에서 CNAME이 적용되지 않는다(##중요)

Storage Account

- **Web Application**에서 **Blob Storage**에 파일 업로드하기
 - **Static Website**를 생성하여 정적 홈페이지를 구성하기
 - Storage account에 Static website를 생성한다
 - **\$web**이라는 Container에 index.html 파일 업로드하기
 - Blob Storage(예: down)를 생성한 후 Access Level을 Container로 변경한다
 - down에 SAS를 생성한다(기간 중요/토큰 복사하기)
 - 해당 Blob storage에 파일 업로드하는 index.html 파일에 Code 작성한다
 - index.html 파일 다운로드:
<https://ysleedown.blob.core.windows.net/azure/index.html>
 - 무작위로 작성된 URL을 복사하여 접속하여 본다
 - 파일 업로드하기
 - 실패한다. CORS에서 업로드를 차단한 것이다
- CORS 설정하기

Settings

- Configuration
- Data Lake Gen2 upgrade
- Resource sharing (CORS)

Blob service | File service | Queue service | Table service

Allowed origins	Allowed methods	Allowed headers	Exposed headers	Max age
https://ysleedown.blob.core.windows.net/azure/index.html ✓	7 selected ✓	x-ms-blob-type, Content-Type ✓	* ✓	9999 ✓
	0 selected ✓			0 ✓

Storage Account

- **Web Application**에서 **Blob Storage**에 파일 업로드하기
 - 파일 업로드하기

File Upload to Azure Blob Storage

1

파일 선택 30년근속-송은영.png

Upload

2

File uploaded successfully!

File name: 1723644847836-30년근속-송은영.png

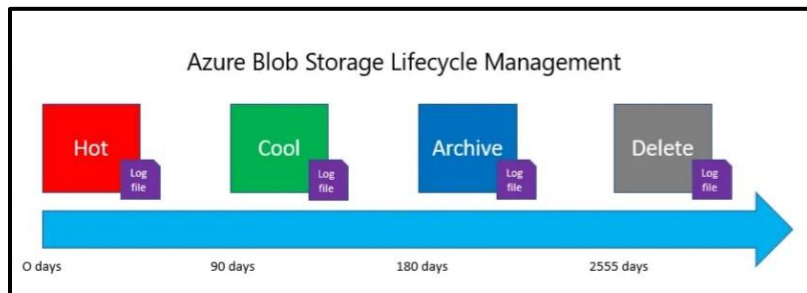
Note: The file is accessible only with proper authentication.

- Blob storage에 파일이 업로드 되었는지 확인하기

Name	Status
☐ 1723640913360-windows_server_2022_logo-removebg-preview....	✓ Current version
☐ 1723642811674-2019년가정사역팀-평화로운엄마아빠.jpg	✓ Current version
☐ 1723644847836-30년근속-송은영.png	✓ Current version
☐ YSLee.jpg	✓ Current version

Storage Account

- 수명 주기 관리(**Lifecycle Management**) 구성하기
 - 수명 주기 관리(Lifecycle Management)는 데이터를 관리하고 비용을 최적화하며 규정 준수를 보장하는 데 중요한 역할을 한다
 - Azure Storage**의 수명 관리는 주로 **Azure Blob Storage**에서 사용되며, 저장된 데이터의 수명 주기를 관리하기 위한 규칙을 설정할 수 있다
 - 수명 관리의 이점
 - 비용 절감: 덜 자주 액세스되는 데이터를 저렴한 저장소 계층으로 이동함으로써 비용을 절감할 수 있다.
 - 데이터 관리 최적화: 자동으로 데이터를 적절한 저장소 계층으로 이동하거나 삭제하여 데이터 관리가 효율적이다.
 - 규정 준수: 특정 기간 후 데이터를 삭제함으로써 데이터 보존 정책을 준수할 수 있다



90일 이상 액세스되지 않은 Blob을 Cool 계층으로 이동하고,
180일 이상 액세스되지 않은 Blob을 Archive 계층으로 이동하며,
2555일 이상 수정되지 않은 Blob을 삭제하는 정책

Storage Account

- 수명 주기 관리(**Lifecycle Management**) 구성하기
 - Lifecycle Management 정책
 - 정책 설정: 수명 주기 정책은 **JSON** 형식으로 작성되며, 각 정책은 **Blob** 데이터의 수명 주기를 관리하기 위한 하나 이상의 규칙으로 구성된다
 - 정책 적용 범위: 해당 **Storage**에 있는 모든 **Blob** 컨테이너 또는 특정 **Blob** 경로에 대해 적용할 수 있다
 - 정책 규칙 구성 요소
 - 조건(**Conditions**): 규칙이 적용되는 조건을 설정한다. 조건에는 **Blob**의 마지막 수정 시간, **Blob** 계층(**Standard, Cool, Archive**) 등이 포함된다
 - 작업(**Actions**): 조건이 충족되었을 때 수행될 작업을 정의한다. 일반적인 작업으로는 **Blob** 계층(**Tier**) 변경, **Blob** 삭제 등이 있다
 - Lifecycle Management 작업
 - **Blob** 계층(**Tier**) 변경: 데이터 액세스 빈도에 따라 **Standard, Cool, Archive** 계층으로 **Blob**을 이동할 수 있다. 예를 들어, 30일 이상 수정되지 않은 **Blob**을 **Cool** 계층으로 이동하거나, 365일 이상 수정되지 않은 **Blob**을 **Archive** 계층으로 이동할 수 있다
 - **Blob** 삭제: 특정 조건이 충족되면 **Blob**을 삭제할 수 있다. 예를 들어, 마지막 수정 후 730일 이상 된 **Blob**을 삭제하는 규칙을 만들 수 있다

Storage Account

- 수명 주기 관리(**Lifecycle Management**) 구성하기
 - Blob 파일 계층(Tier) 변경하기
 - Azure Blob Storage에서 Blob 계층(Hot, Cool, Archive)은 Blob 또는 Blob 컨테이너가 생성된 후에도 지정하고 수정할 수 있다. 각 계층은 저장 비용과 액세스 패턴에 따라 다르다
 - **Hot Tier:**
 - 데이터가 자주 액세스되는 경우에 사용한다
 - 저장 비용이 가장 높지만, 액세스 비용이 낮다
 - **Cool Tier:**
 - 데이터가 자주 액세스되지 않지만, 여전히 즉시 액세스가 필요한 경우에 사용한다
 - 저장 비용이 Hot Tier보다 저렴하며, 액세스 비용은 더 높다
 - **Archive Tier:**
 - 데이터가 거의 또는 전혀 액세스되지 않으며, 장기 보관이 필요한 경우에 사용한다
 - 저장 비용이 가장 저렴하지만, 액세스 비용이 높고 액세스하기 위해 복원 시간이 필요하다

Storage Account

- 수명 주기 관리(**Lifecycle Management**) 구성하기
 - Blob 파일 계층(Tier) 변경하기

Name	Modified	Access tier
☐  엄마와 마음이.png	8/9/2024, 8:43:47 AM	Hot (Inferred)
☐  A.I 시대 인문학 가치-3pdf.pdf	8/9/2024, 8:43:24 AM	Hot (Inferred)

> publicdownload >

A.I 시대 인문학 가치-3pdf.pdf ...

Blob

 Save  Discard  Download  Refresh  Delete  Change tier  Acquire lease

Change tier ×

A.I 시대 인문학 가치-3pdf.pdf

Optimize storage costs by placing your data in the appropriate access tier. [Learn more](#)

Access tier

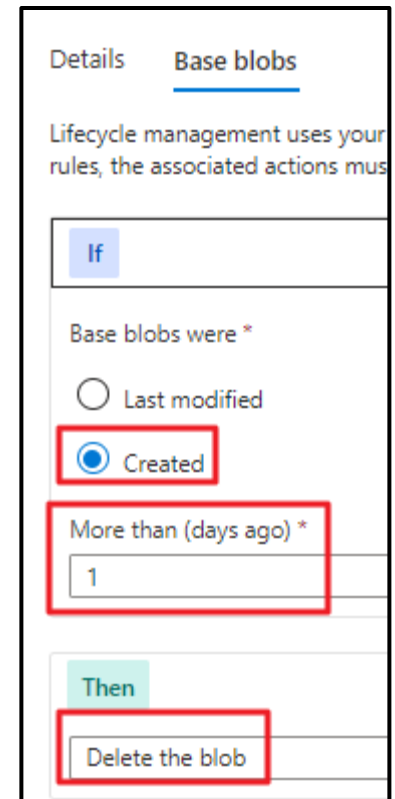
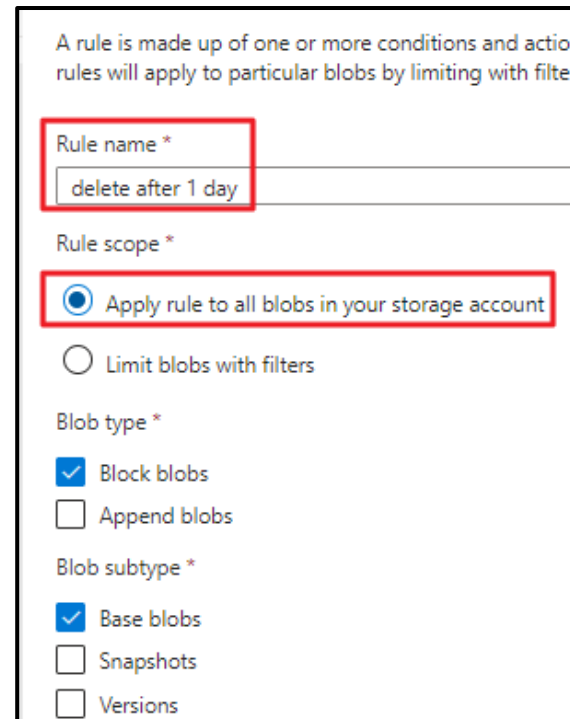
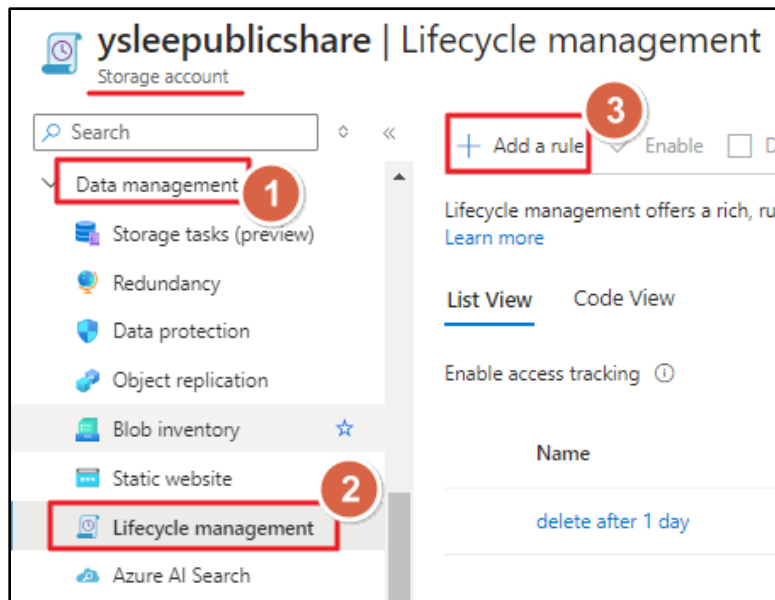
▼

- Hot (Inferred)
- Cool**
- Cold
- Archive

Name	Modified	Access tier
☐  엄마와 마음이.png	8/9/2024, 8:43:47 AM	Hot (Inferred)
☐  A.I 시대 인문학 가치-3pdf.pdf	8/9/2024, 8:43:24 AM	Cool

Storage Account

- **Demo-수명 주기 관리-1(생성된 Blob 처리하기)**
 - Blob 파일이 생성된 후 24시간이 지나면 자동으로 삭제하기
 - 최소 시간이 24시간이 지나야 한다(1일)
 - 파일을 공유한 후 24시간이 지나면 자동으로 삭제하고자 할 때 유용하다



Storage Account

- **Demo-수명 주기 관리-2(저장 후 Access하지 않은 Blob 처리)**
 - 30일 이상 액세스되지 않은 Blob을 Cool 계층으로 이동하고, 365일 이상 액세스되지 않은 Blob을 Archive 계층으로 이동하며, 730일 이상 수정되지 않은 Blob을 삭제하는 정책 생성
 - 첫 번째 규칙:
 - 30일 이상 액세스되지 않은 Blob을 Cool 계층으로 이동
 - Rule name: **MoveToCool**
 - 두 번째 규칙:
 - 365일 이상 액세스되지 않은 Blob을 Archive 계층으로 이동
 - Rule name: **MoveToArchive**
 - 세 번째 규칙:
 - 730일 이상 수정되지 않은 Blob을 삭제
 - Rule name: **DeleteBlob**

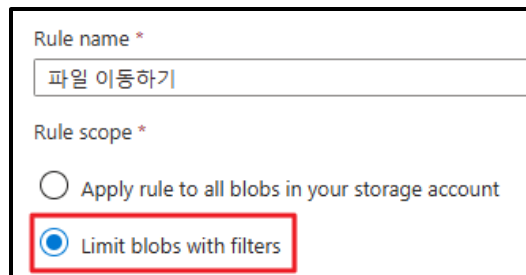
The screenshot shows the 'Base blobs' configuration page for a lifecycle management rule. The 'If' section is expanded, showing the condition 'Base blobs were *'. Under this, 'Last modified' is selected with a radio button, and 'Created' is unselected. Below this, the 'More than (days ago) *' field is set to 30. The 'Then' section is also expanded, showing the action 'Move to cool storage'.

Storage Account

- **Demo-수명 주기 관리-3(Blob prefix 사용)**

- **Filter**를 사용하여 특정한 Container의 Blob에게만 적용하기

- **Limit blobs with filters** 선택



Rule name *

파일 이동하기

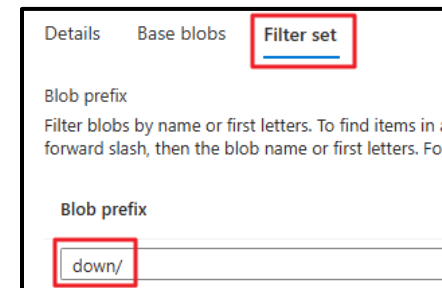
Rule scope *

☐ Apply rule to all blobs in your storage account

☒ Limit blobs with filters

- 특정한 Container(예: down)에 저장된 모든 Blob을 필터링하여 적용하기

- **container_name/**



Details Base blobs **Filter set**

Blob prefix

Filter blobs by name or first letters. To find items in a forward slash, then the blob name or first letters. For

Blob prefix

down/

- 특정 하위 경로 아래의 Blob만 필터링하기

- **container_name/subfolder/**

- 접두사로 특정 Blob만 필터링하기

- **container_name/report**

- report로 시작하는 Blob만 필터링된다 (예: report_2024.pdf, report_Q3.xlsx)

Storage Account

- **Demo-수명 주기 관리-4(Blob index match 사용)**
 - 각 **Blob**에 지정한 **Metadata**로 **Filtering**하기
 - 먼저, 특정 **Container**에 저장된 각 **Blob**에 메타데이터 추가하기
 - 먼저 업로드된 각 **Blob** 파일을 클릭하여 아래로 스크롤한다
 - Key-Value 쌍을 입력한다(##한글 사용 안됨)
 - Key: **Category** / Value: **Images**
 - Key: **CreatedBy** / Value: **Admin**

Key	Value
Category	Pdf
CreatedBy	Admin

- 추가한 Rule의 **Filter set**에서 Blob index match 설정하기

Blob index match

If you have indexed items in containers with keys and values, you can filter for them.

Key	Value
Category *	Pdf

실습:

- 전체 실습 내용
 1. **Storage Account** 생성 및 기본 설정
 2. **Blob Storage** 활용
 3. **File Share** 연결
 4. **Static Website** 호스팅
 5. **Lifecycle Management** 정책 구성

1. Storage Account 생성 및 기본 설정

- 목표
 - Azure Storage Account를 생성하고 기본 속성을 이해한다.
 - 복제 옵션, 성능 계층, 엔드포인트 URL을 확인한다.
- 실습 단계
 - 1) Storage Account 생성 절차
 - ① Azure Portal 로그인 → 좌측 메뉴에서 Storage accounts 선택 → + 만들기(Create) 클릭
 - ② 기본 설정(Basics)
 - 구독(Subscription): 기본 구독 선택
 - 리소스 그룹(Resource Group): myRG (없으면 새로 생성)
 - Storage account 이름: 전 세계적으로 고유한 소문자 영문+숫자 조합 (예: mystorage12345)
 - 지역(Location): East US

1. Storage Account 생성 및 기본 설정

- 실습 단계

- 1) Storage Account 생성 절차

- ② 기본 설정(Basics)

- 성능(Performance):
 - Standard: 범용, 비용 효율적 (Blob, File, Queue, Table 지원)
 - Premium: 고성능, 낮은 지연 시간 (특정 워크로드용)
- 계정 종류(Account kind):
 - StorageV2 (general purpose v2) 선택 (가장 최신, 권장)
- 복제 옵션(Replication):
 - LRS (Locally-redundant storage): 기본값, 단일 지역 내 3개 복제
 - ZRS, GRS, RA-GRS 등은 고가용성/재해 복구 필요 시 선택

- ③ 고급(Advanced) (선택)

- 보안: 기본적으로 Microsoft 관리형 키로 암호화
- 필요 시 고객 관리형 키(Customer-managed key)로 변경 가능

1. Storage Account 생성 및 기본 설정

- 실습 단계

- 1) Storage Account 생성 절차

- ③ 고급(Advanced) (선택)

- 보안: 기본적으로 Microsoft 관리형 키로 암호화
- 필요 시 고객 관리형 키(Customer-managed key)로 변경 가능

- ④ 네트워킹(Networking)

- 퍼블릭 엔드포인트: 기본값 → 인터넷에서 접근 가능
- 필요 시 특정 VNet/Subnet 또는 IP만 허용 설정 가능

- ⑤ 데이터 보호(Data protection)

- Soft delete(Blob, File) 옵션을 켜면 삭제된 데이터 복구 가능

- ⑥ 검토 + 만들기(Review + Create) → 검증 후 만들기(Create) 클릭

1. Storage Account 생성 및 기본 설정

- 실습 단계

- 2) 생성 후 확인 작업

- ① 배포 완료 후 → 리소스로 이동(Go to resource)

- ② 개요(Overview) 탭 확인

- Storage account 이름, 위치, 복제 옵션, 성능 계층

- 엔드포인트(Endpoints):

- Blob: <https://<account>.blob.core.windows.net/>

- File: <https://<account>.file.core.windows.net/>

- Queue: <https://<account>.queue.core.windows.net/>

- Table: <https://<account>.table.core.windows.net/>

- ③ 엔드포인트 URL을 브라우저에서 열어보기 (권한 없으므로 바로는 접근 불가 → 나중에 컨테이너/파일 생성 후 확인 가능)

- 확인 포인트

- 계정 이름은 전역 고유해야 하므로 중복 불가

- 복제 옵션(LRS, ZRS, GRS, RA-GRS, GZRS 등)에 따라 가용성과 비용이 달라짐

- 생성 직후 Storage Account는 빈 껍데기 리소스이며, 실제로는 Blob/File/Queue/Table을 만들어야 데이터 저장 가능

2. Blob Storage 활용

- 목표
 - Blob Container를 생성한다.
 - Access Level(Private / Blob / Container)을 변경하고 차이를 체험한다.
 - Blob 파일 업로드, 다운로드, 공유를 실습한다.
- 실습 단계
 - 1) 컨테이너 생성
 - ① Azure Portal → Storage accounts → 생성한 계정 선택
 - ② 왼쪽 메뉴 → Containers → + Container 클릭
 - ③ 이름 입력: 예) mycontainer
 - 소문자만 가능, 고유해야 함
 - ④ 공개 액세스 수준(Public access level) 선택
 - Private (기본값): 인증된 사용자만 접근 가능
 - Blob: 익명 사용자도 Blob URL만 알면 접근 가능 (권장)
 - Container: 익명 사용자도 컨테이너 안의 모든 Blob 목록 접근 가능
 - ⑤ 만들기(Create) 클릭

2. Blob Storage 활용

- 실습 단계

2) Blob 업로드

- ① 생성한 컨테이너(mycontainer) 클릭
- ② 상단 메뉴 → Upload
- ③ 로컬 PC에서 이미지/문서 파일 선택 (예: sample.jpg)
- ④ 업로드 완료 후 Blob 목록에서 파일 확인

3) Blob Access Level 변경 및 테스트

- ① Blob 클릭 → URL 확인
 - `https://<storage_account>.blob.core.windows.net/mycontainer/sample.jpg`
- ② Access Level에 따른 동작 차이 확인
 - Private: URL 접속 시 권한 오류 발생 (SAS 토큰 필요)
 - Blob: URL만 알면 다운로드 가능 (목록은 못 봄)
 - Container: URL만 알면 컨테이너 안 모든 Blob 목록 조회 가능

2. Blob Storage 활용

- 실습 단계
 - 4) SAS(Shared Access Signature) URL 발급
 - ① Blob 선택 → Generate SAS
 - ② 권한(Read, Write, List, Delete 등)과 만료 시간 설정 (예: 1시간 후 만료)
 - ③ 생성된 SAS URL 복사 후 브라우저에서 접속
 - Private 상태여도 SAS URL을 사용하면 접근 가능
 - 5) Blob 유형 실습 (선택)
 - ① Blob은 Block Blob / Append Blob / Page Blob 3가지 유형이 있음
 - Block Blob: 일반 파일 저장 (이미지, 동영상 등)
 - Append Blob: 로그 기록 등 지속적 추가 데이터
 - Page Blob: VM 디스크 저장소
- 확인 포인트
 - Container Access Level에 따라 보안 vs 공유 편의성 차이 발생
 - SAS URL을 통해 Private Blob도 제한적으로 공유 가능
 - Blob 유형에 따라 사용 사례 다름: 파일 저장 / 로그 / VM 디스크

3. File Share 연결

- 목표
 - Azure Storage 계정에서 File Share를 생성한다.
 - Windows VM에서 네트워크 드라이브로 연결한다.
 - 업로드/다운로드를 테스트한다.
- 실습 단계
 - 1) File Share 생성
 - ① Azure Portal → Storage accounts → 원하는 스토리지 계정 선택
 - ② 좌측 메뉴 → File shares → + File share 클릭
 - ③ 설정 입력:
 - 이름: myshare
 - 계층(Quota): 예) 100 GB
 - 성능: Standard (일반용), 필요시 Premium 선택
 - ④ 만들기(Create) 클릭
 - 2) Windows VM 준비
 - ① 동일 구독/리소스 그룹 내 Windows VM 생성 (예: win-vm01)
 - ② VM에 RDP(Remote Desktop)로 접속

3. File Share 연결

- 실습 단계

3) File Share 연결 (네트워크 드라이브 매핑)

- ① Storage 계정 → File shares → 생성한 myshare 선택
- ② 상단 메뉴에서 연결(Connect) 클릭
- ③ OS 선택에서 Windows 선택 후 복사
- ④ VM의 PowerShell에서 해당 스크립트 실행
- ⑤ 파일 탐색기에서 Z: 드라이브 생성 확인

4) 파일 업로드/다운로드 테스트

- ① VM에서 Z: 드라이브 열기
- ② 로컬에서 텍스트 파일(test.txt) 복사 → Z:에 붙여넣기
- ③ Azure Portal의 File Share → Blob 목록 확인 → test.txt 업로드 완료 확인
- ④ 반대로 Portal에서 파일 업로드 후 VM에서 다운로드 확인

3. File Share 연결

- 실습 단계

- 5) 네트워크 환경 관련 주의사항

- ① **SMB 445** 포트는 일부 국가/ISP에서 막혀 있음 → **VM** 내부에서 연결해야 정상 동작
 - ② 따라서 물리적인 **PC**에서 직접 연결은 제한적, 반드시 **VM**을 통해 연결

- 확인 포인트

- **File Share**는 **SMB** 프로토콜을 이용 → **Windows** 탐색기에서 네트워크 드라이브처럼 사용 가능
 - **Azure Backup**과 연계 가능 (자동 백업 옵션 존재)
 - **VM**에서 **Z:** 드라이브에 올린 파일이 **Portal** → **File Share** 화면에서도 동일하게 보이면 성공

4. Static Website 호스팅

- 목표
 - Storage Account의 Static Website 기능을 활성화한다.
 - \$web 컨테이너에 정적 파일(index.html 등)을 업로드한다.
 - 엔드포인트 URL을 통해 웹사이트를 확인한다
- 실습 단계
 - 1) Static Website 기능 활성화
 - ① Azure Portal → Storage accounts → 대상 계정 선택
 - ② 좌측 메뉴 → Static website 선택
 - ③ 상태(Status): Enabled 선택
 - ④ Index document name: index.html
 - ⑤ Error document path: 404.html (선택)
 - ⑥ 저장(Save) 클릭 → Storage 계정에 \$web 컨테이너 자동 생성

4. Static Website 호스팅

- 실습 단계
 - 2) 정적 파일 준비
 - ① 로컬 PC에서 index.html 생성 (예시):

```
<!DOCTYPE html>
<html>
<head>
  <title>Azure Static Website</title>
</head>
<body>
  <h1>Hello from Azure Storage Static Website!</h1>
  <p>This is a simple static page hosted in Blob Storage.</p>
</body>
</html>
```

- 3) \$web 컨테이너에 업로드
 - ① Storage 계정 → Containers → \$web 선택
 - ② 상단 Upload 클릭 → index.html 파일 업로드
 - ③ 필요 시 404.html도 함께 업로드

4. Static Website 호스팅

- 실습 단계
 - 4) 엔드포인트 URL 확인
 - ① Static Website 설정 화면에서 Primary endpoint 확인
 - 예: `https://<storage_account>.z12.web.core.windows.net/`
 - ② 브라우저에서 접속 → `index.html` 내용 출력 확인
- 확인 포인트
 - \$web 컨테이너는 Static Website 전용, 삭제 시 웹사이트도 접근 불가
 - HTML/CSS/JS/이미지 등 정적 콘텐츠만 제공 가능
 - PHP, Node.js 등 서버사이드 실행 불가
 - 매우 저렴하고 간단하게 웹사이트를 배포할 수 있음

5. Lifecycle Management 정책 구성

- 목표
 - Azure Blob Storage에 저장된 데이터의 수명주기 정책을 자동화한다.
 - 오래된 데이터를 저비용 계층(**Cool, Archive**)으로 이동시키거나, 필요 시 자동 삭제하는 규칙을 설정한다
- 실습 단계
 - 1) Lifecycle Management 메뉴 이동
 - ① Azure Portal → Storage accounts → 대상 계정 선택
 - ② 좌측 메뉴 → Data management → Lifecycle Management 클릭
 - 2) 새 규칙 생성
 - ① 상단 + Add a rule 클릭
 - ② 규칙 이름 입력 (예: MoveToCool)
 - ③ 범위 선택:
 - Entire storage account (전체 컨테이너 적용)
 - 또는 Limit blobs with filters → 특정 컨테이너/폴더/접두사(prefix)로 제한 가능

5. Lifecycle Management 정책 구성

- 실습 단계

- 3) 조건(Conditions) 설정

- ① 조건 기준:

- Last modified (마지막 수정 시점)
 - Last accessed (마지막 접근 시점)
 - Blob 유형 (Block Blob / Append Blob / Page Blob)

- ② 예시:

- 30일 이상 수정되지 않은 Blob → Cool 계층으로 이동
 - 365일 이상 접근하지 않은 Blob → Archive 계층 이동
 - 730일 이상 수정되지 않은 Blob → 삭제

5. Lifecycle Management 정책 구성

- 실습 단계

- 4) 작업(Actions) 설정

- ① 작업 선택 (하나 이상 가능):

- Move to cool storage
 - Move to archive storage
 - Delete blob

- ② 예시 규칙 구성:

- Rule 1: MoveToCool → 30일 이상 수정되지 않은 Blob → Cool Tier 이동
 - Rule 2: MoveToArchive → 365일 이상 접근하지 않은 Blob → Archive Tier 이동
 - Rule 3: DeleteOldBlob → 730일 이상 수정되지 않은 Blob → 삭제

5. Lifecycle Management 정책 구성

- 실습 단계
 - 5) 규칙 적용 및 확인
 - ① Add → 규칙 생성 완료
 - ② 정책은 비동기적 적용 → Blob이 조건에 부합할 때 자동 실행됨
 - ③ Blob 속성에서 Access Tier 변경 이력을 통해 정책 동작 여부 확인
 - 6) (고급) 필터링 실습
 - ① Blob prefix 사용: 특정 컨테이너/경로에만 적용 (예: report/ 접두사)
 - ② Blob index match 사용: 메타데이터 기반 필터링 (예: Category=Images)
- 확인 포인트
 - Lifecycle 정책은 Blob Storage 전용 기능 (File, Queue, Table에는 적용 안 됨)
 - Access Tier 변경 시 저장 비용과 읽기/복원 비용 차이가 발생 → 주의 필요
 - 규정 준수를 위해 데이터 보존 정책을 자동화할 수 있음